

DevOps Notes — Beginner Friendly

(Developer + Operations)

What is DevOps?

DevOps is a combination of **Development** (writing code) and **Operations** (managing servers and software).

The goal is to make it easier to build, test, and deliver software faster.

SESSION 01

PART 1 — Setting Up Your Computer

What is Chocolatey?

Imagine you want to install apps on your phone — you go to the Play Store or App Store and just click install.

Chocolatey is the same thing, but for your Windows computer using the Command Prompt (CMD).

Instead of going to websites and downloading installers one by one, you just type a command and it installs automatically.

How to use it:

First, open **CMD or PowerShell as Administrator** (right-click → Run as Administrator).

This is important — without admin mode, Chocolatey won't work.

Commands to install the tools we need:

...

choco --version

...

> This checks if Chocolatey is installed. If you see a version number, it's working.

...

```
choco install virtualbox -y
choco install vagrant -y
choco install git -y
choco install vscode -y
...
```

> The `-y` at the end means "yes, install without asking me to confirm."

PART 2 — Understanding Virtual Machines

What is a Virtual Machine (VM)?

A **Virtual Machine** is like a computer inside your computer.

For example — your laptop runs Windows. But inside it, you can create a separate "fake computer" that runs Linux. This fake computer is called a Virtual Machine. It has its own operating system, files, and settings — completely separate from your real computer.

Why is this useful?

- Developers use VMs to test software in different environments
- Companies use VMs to run many servers on one physical machine
- If the VM breaks, your real computer is safe

What is a Hypervisor?

A **Hypervisor** is the software that creates and manages Virtual Machines.

There are 2 types:

Type 2 Hypervisor — For Personal Laptops

- This runs ON TOP of your existing operating system (like Windows or Mac)
- Think of it like an app you install on your laptop
- It can only create a limited number of VMs because it's sharing resources with your OS
- Example: **Oracle VirtualBox** (the one we installed)

--- ****Type 1 Hypervisor — For Big Companies (Bare Metal)****

- This runs DIRECTLY on the hardware — there is no existing OS underneath
- It has access to ALL the computer's RAM and power
- Can create many more VMs — used by companies like Amazon, Google for their data centers
- Example: ****VMware ESXi****

--- **### What is Vagrant?**

Vagrant is a tool that works WITH VirtualBox to ****automatically set up virtual machines**** for you.

Instead of manually clicking through settings to create a VM, you write simple instructions in a file and Vagrant creates it for you automatically.

--- **## PART 3 — Git Basics**

What is Git?

****Git is a tool that tracks changes to your files over time.****

Think of it like this — imagine you are writing an essay and you save different versions:

- essay_v1.docx
- essay_v2.docx
- essay_final.docx
- essay_final_REAL.docx

Git does this automatically and much more neatly. Every time you save (called a "commit"), Git remembers the exact state of all your files.

--- **### What is GitHub?**

****GitHub is a website where you store your Git projects online.****

Git = works on your computer (local)

GitHub = stores your project on the internet (remote/cloud)

It's like Google Drive, but specifically designed for code projects.

Opening Git Bash

Git Bash is a terminal (command window) that lets you run Git commands.

To open it:

1. Go to any folder
2. Right-click
3. Click "More options"
4. Click "Git Bash"

Basic Navigation Commands

These commands help you move around folders in Git Bash:

...

pwd

...

> ****Print Working Directory**** — tells you which folder you are currently in.

> Example output: /c/Users/afsha/OneDrive/Desktop

...

mkdir internship

...

> ****Make Directory**** — creates a new folder called "internship" on your Desktop.

...

cd internship

...

> ****Change Directory**** — goes inside the "internship" folder.

> (like double-clicking a folder to open it)

...

```
cd ..
```

```
...
```

> Goes BACK one folder (goes to the parent folder).

```
...
```

```
ls
```

```
...
```

> **ls** — shows all files and folders inside your current location.

```
...
```

```
ls -l
```

```
...
```

> Same as `ls` but shows each file on its own line (easier to read).

```
---
```

Creating Files

```
...
```

```
touch abc.txt
```

```
...
```

> Creates an **empty file** called abc.txt.

> (like right-clicking → New → Text Document, but faster)

```
...
```

```
cat > file1.txt
```

```
...
```

> Creates a file AND lets you type content inside it.

> After typing everything, press **Ctrl + D** to save and exit.

```
---
```

Starting Git in a Folder

```
...
```

```
git init .
```

```
...
```

> This tells Git — "start tracking this folder."

> The dot (.) means "here, in this current folder."

> After running this, you'll see **(master)** appear — that means you're on the main branch.

The 3-Step Git Process (Add → Commit → Push)

Every time you want to save and upload your work, you follow these 3 steps:

****Step 1: Check what's changed****

```
git status
```

> Shows which files are new or changed.

> New files that Git doesn't know about yet are called "untracked files."

****Step 2: Stage your files (tell Git which files to save)****

```
git add -A
```

> The ``-A`` means "add ALL files."

> This moves your files to a "staging area" — like putting items in a box before shipping.

After running this, do ``git status`` again — the "untracked" message should be gone.

****Step 3: Commit (officially save a snapshot)****

```
git commit -m "Internship-01 file"
```

> This takes a permanent snapshot of all your staged files.

> The ``-m`` lets you add a message describing what you changed.

> Always write something meaningful so you remember what this commit was about.

> ****Important:**** Without doing Add and Commit, you CANNOT push (upload) your files to GitHub!

Why Linux for Servers?

You might wonder — why do companies use Linux instead of Windows on their servers?

- **No license fee** — Linux is free, Windows is not
- **Cheaper subscriptions** for server hosting
- More control and customizable

SESSION 02

PART 4 — Connecting to GitHub

Creating a Personal Access Token

GitHub needs a **token** (like a password) to verify it's really you when you upload from Git Bash.

How to create one:

1. Go to GitHub → click your profile photo
2. Click **Settings**
3. Scroll down → click **Developer Settings**
4. Click **Personal Access Tokens**
5. Click **Generate New Token** → **Classic Token**
6. Copy the token and **save it somewhere safe** — you won't be able to see it again!

Connecting Your Local Folder to GitHub

After creating a new repository on GitHub, it shows you 3 commands.

Copy those commands and run them in Git Bash — this connects your local folder to GitHub.

Save Your Token So Git Doesn't Keep Asking

...

```
git config --global credential.helper store
```

```
...
```

> After running this once, Git will remember your token and stop asking for it every time.

```
---
```

PART 5 — Working with Multiple Folders

Now let's see how to work with folders inside our project:

```
...
```

```
mkdir session1
```

```
mkdir session2
```

```
...
```

> Creates two folders inside your project.

```
...
```

```
ls
```

```
...
```

> You should now see: abc.txt file1.txt session1/ session2/

```
...
```

```
cd session1
```

```
...
```

> Go inside session1.

```
...
```

```
touch recording.txt
```

```
...
```

> Create a file inside session1.

```
...
```

```
cd ..
```

```
...
```

> Go back to the main project folder.

```
...
```

```
cd session2
```

```
...
```

> Go inside session2.

...

touch recording.txt

...

> Create a file inside session2 too.

...

cd ..

...

> Go back to main folder.

...

git status

...

> Now Git will show session1 and session2 as untracked.

...

git add -A

git commit -m "Session Recording"

git push

...

> Stage, commit, and upload everything to GitHub.

PART 6 — Feature Branches

What is a Branch?

Imagine you are working on a book and you want to try a new ending without ruining the original.

You make a **copy** of the book, write the new ending on the copy, and if you like it, you merge it back into the original.

In Git, this copy is called a Branch.

- The original is called the **main** branch

- Your copy is called a **feature branch**

This way, multiple developers can work on different features at the same time without breaking each other's work.

Creating a Feature Branch

...

```
git checkout -b feature/testing
```

...

> Creates a new branch called "feature/testing" AND switches to it.

> You'll see: "Switched to a new branch 'feature/testing'"

...

```
git branch -a
```

...

> Shows all branches you have locally.

...

```
git checkout main
```

...

> Switches back to the main branch.

...

```
git checkout feature/testing
```

...

> Switches back to your feature branch.

Important Thing to Understand About Branches

When you are in the **feature/testing** branch, you can only see files that were created in that branch.

For example:

- In **feature/testing** → you can see: abc.txt, file1.txt, sample1, sample2

- In **feature/testing** → you CANNOT see: session1, session2 (those are in main)

Each branch has its own separate world of files!

Getting the Latest Files from Main into Your Branch

If someone else added files to the main branch and you want those files in your feature branch too:

...

```
git pull origin main
```

...

- > This pulls everything from main INTO your current feature branch.
- > Now your feature branch has everything from main PLUS your own new files.

Pushing a Feature Branch to GitHub for the First Time

The first time you push a new branch, you need to tell Git where to send it:

...

```
git push --set-upstream origin feature/testing
```

...

- > After this, you can just use `git push` normally.

After this, go to GitHub — you'll be able to see your feature/testing branch there!

What is a Pull Request?

A **Pull Request** (PR) is a way to say:

"Hey, I finished my work on the feature branch — can someone review it and merge it into main?"

It's like submitting your work for review before it becomes official.

How to do it on GitHub:

1. Go to GitHub → click **"Pull Requests"**
2. Set the title as: feature/testing → main
3. Click **"Confirm Pull"**
4. A reviewer (teammate or senior dev) looks at the code
5. If they say OK → the merge happens
6. Now everything from feature/testing is in main!

FULL SUMMARY — The Complete Workflow

Here is the complete workflow every time you work on something new:

...

Step 1: `git checkout -b feature/backend`
(Create and switch to a new feature branch)

Step 2: `git pull origin main`
(Get the latest files from main)

Step 3: Create your files / write your code
e.g. `touch sample..`

Step 4: `git status`
(See what changed)

Step 5: `git add -A`
(Stage all changes)

Step 6: `git commit -m "new sample"`
(Save a snapshot with a message)

Step 7: `git push`
(Upload to GitHub)

Step 8: On GitHub → Create a Pull Request → Merge into main
...

> Files go: Your Computer → Local Git Repo → GitHub (feature branch) → Main Branch

End of Notes — Sessions 01 & 02